

Rapport Projet Compilation L3 2025

Ben Hamouda Maïssa
Gavau—Pelissier Liam

Sommaire :

1. Introduction
2. Interface utilisateur
3. Architecture
4. Choix d'implémentation
5. Problèmes rencontrés
6. Conclusion

1. Introduction

L'objectif de ce projet est la conception et l'implémentation d'un compilateur complet pour le langage TPC, un sous-ensemble du C. Ce projet s'appuie directement sur le projet d'analyse syntaxique du premier semestre que nous avons intégré.

Le compilateur produit prend en entrée un programme TPC valide et génère du code assembleur NASM au format elf64 bits, conforme aux conventions d'appel AMD64. Il effectue également une analyse sémantique complète et émet des messages d'erreur ou d'avertissement.

2. Interface utilisateur

Le compilateur tpcc accepte les options suivantes :

- `-t`, `--tree` : affiche l'arbre syntaxique abstrait sur la sortie standard
- `-s`, `--syntabs` : affiche toutes les tables de symboles (globale et locales par fonction)
- `-h`, `--help` : affiche l'aide et quitte

Le programme se compile via make depuis le répertoire du projet. Il peut être lancé de deux fonctions :

- `./bin/tpcc [OPTIONS] < FILE.tpc`
- `./bin/tpcc [OPTIONS] FILE.tpc`

Le fichier assembleur produit est nommé `_anonymous.asm` si le fichier est dans l'entrée standard sinon `FILE.asm` si le fichier `.tpc` est passé en argument.

Pour lancer notre banc de test, il faut :

```
make
gcc -o scripTest scriptTest.c
./scriptTest
```

Pour lancer le compilateur et compiler le `.asm` :

```
make run TPC=FILE.tpc
```

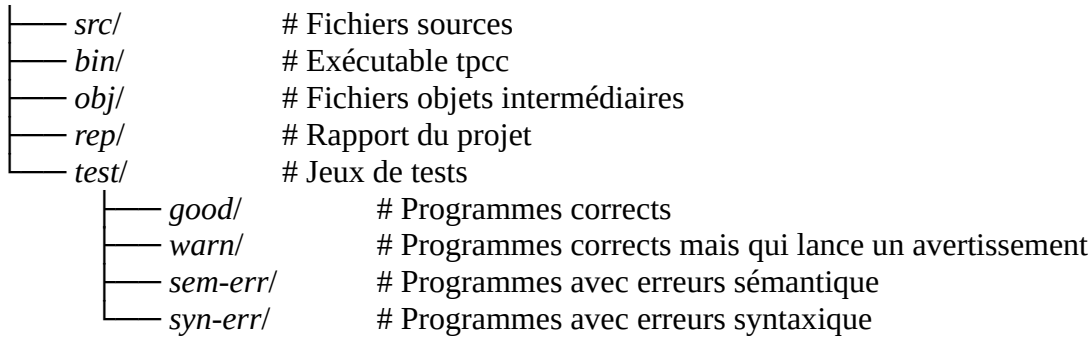
Les valeurs de retour sont :

- 0 : programme source correct (même s'il contient des avertissements)
- 1 : erreur lexicale ou syntaxique
- 2 : erreur sémantique
- 3 ou plus : erreur de la ligne de commande, fichier introuvable ou autre

3. Architecture

Le projet est organisé selon la structure demandée :

ProjetCompilationL3_BEN-HAMOUDA_GAVAU—PELISSIER/



Les composants principaux sont :

- *lex_proj.lex* : analyseur lexical généré par flex, reconnaît les lexèmes du langage TPC (mots-clés, identifiants, opérateurs, littéraux, commentaires)
- *bison_proj.y* : analyseur syntaxique généré par bison, vérifie la grammaire TPC et construit l'arbre abstrait
- *tree.c* / *tree.h* : module de gestion de l'arbre syntaxique abstrait
- *compiler.c* / *compiler.h* : gestion des tables des symboles (création, insertion, recherche, affichage, libération)
- *struct_table.c* / *struct_table.h* : table des structures, champs, et informations de type (*TypeInfo*)
- *semantique.c* / *semantique.h* : analyse sémantique et génération de code NASM en un seul parcours de l'arbre

4. Choix d'implémentation

I. Tables des symboles

Chaque fonction possède sa propre table des symboles locale (paramètres et variable locales), distincte de la table globale. Les entrées portent un champs *kind* ('v' pour variable, 'f' pour fonction) ainsi qu'un offset qui indique la position de la variable sur la pile locale (déplacement par rapport a *rbp*).

Le calcul de l'offset est cumulatif : à chaque nouvelle variable locale, on parcourt la table pour déterminer l'offset maximum et on ajoute la taille de la variable. La taille minimale allouée est de 4 octets pour garantir un alignement de base.

II. Tables des structures

Les définitions de structures sont stockées dans une table globale séparée (*struct_table.c*). Chaque entrée *StructDef* contient la liste chaînée de ses champs (*Field*), chacun portant son nom, son type et éventuellement le nom du type structure imbriqué. La taille d'une structure est calculée récursivement en sommant les tailles de ses champs.

III. Génération de code NASM

La génération de code est intégrée au parcours sémantique : pour chaque construction du langage, le code assembleur correspondant est émis directement dans le fichier de sortie.

La stratégie de génération repose sur une pile d'évaluation : toutes expression pousse son résultat sur la pile, et les opérateurs dépilent leurs opérandes. Cela simplifie la gestion des expression imbriquées et des appels de fonctions.

Points notables :

- Patch de la taille du frame : au début de chaque fonction, on émet un *sub rsp, placeholder* en enregistrant la position dans le fichier via *ftell()*. Après avoir analyser tout le corps de la fonction et calculé l'espace réel nécessaire (arrondi au multiple de 16), on revient à cette position via *fseek()* pour écrire la vraie valeur. Cela évite d'avoir à effectuer deux passes sur l'arbre.
- Convention AMD64 : les arguments sont passés dans *rdi*, *rsi*, *rdx*, *rcx*, *r8*, *r9*. En entrée de la fonction, ces registres sont sauvegardés en mémoire locale dès le prologue. A l'appel, les arguments sont d'abord évalués et empilés, puis chargés dans les registres dans l'ordre inverse.
- Variables globales : elles sont déclarées dans une section *.bss* (non initialisées) générée après le code, en utilisant *resd* (int), *resb* (char) ou *resb N* (struct de taille N).

IV. Fonctions built-in

Les quatre fonctions d'entrées-sorties (*putint*, *putchar*, *getint*, *getchar*) sont écrites entièrement en NASM et ajoutées en fin de fichier assembleur uniquement si elles sont utilisées dans le programme source (grâce à des drapeaux *need_putint*, *need_putchar*, etc).

5. Difficultés rencontrées

Gestion de la taille du frame de pile

La principale difficulté technique a été de déterminer la taille du frame local avant de l'avoir complètement analysé. En NASM, l'instruction *sub rsp, N* doit apparaître dans le prologue, mais la valeur N n'est connue qu'après avoir parcouru toutes les déclarations locales.

Solution : nous utilisons la technique du patch différé. On écrit *sub rsp, 00000000h* comme placeholder, on enregistre la position dans le fichier avec *ftell()*, puis on analyse le corps. En fin de fonction, on calcule la taille réelle et on revient écrire la valeur exacte avec *fseek()*. Cette approche nous a évité d'implémenter une passe séparée de calcul d'offsets.

Alignement de pile et conventions AMD64

Respecter scrupuleusement l'alignement sur 16 octets requis par l'ABI AMD64 avant chaque instruction *call* s'est avéré délicat, notamment dans le cas d'appels imbriqués ou avec un nombre variable d'arguments. Nous avons choisi d'arrondir systématiquement la taille du frame au multiple de 16 supérieur ($\text{frame_size} = (\text{max_off} + 15) \& \sim 15$) pour garantir cet alignement en entrée de fonction.

Chose pas implémenter

- Vérification du nombre et du type des arguments lors des appels de fonctions :
Implémenter cela nécessiterait de stocker la liste typée des paramètres dans la table des symboles, ce qui représentait un refactoring important en fin de projet.
- Warning int -> char pour les paramètres de fonctions (le warning est bien émis pour les affectations simples, mais pas lors du passage d'argument).
- Génération de code pour le retour d'une valeur de type structure (return d'un struct) :
la vérification sémantique est présente mais la generation de code n'est pas implémenté.
- Génération de code pour l'accès et l'affectation de champs de structures locales

6. Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble de la chaîne de compilation : de l'analyse lexicale et syntaxique héritée du premier semestre, jusqu'à la génération de code assembleur fonctionnel pour une architecture réelle. La gestion des structures, des conventions d'appel AMD64 et de la table des symboles avec offsets ont constitué les défis les plus formateurs.

Le résultat est un compilateur fonctionnel qui couvre les principales constructions du langage TPC et produit du code NASM assemblable et exécutable.